



Centro de Enlace Universitario
Felipe Villanueva
Ciencia con Valores

TRABAJO FINAL

SQL Práctica-Aplicada con Componente Teórico

Bases de Datos Avanzado

Estudiante:	Humberto Zambrano Carreño
Materia:	Base de Datos Avanzados B
Trabajo:	Trabajo Final
Fecha:	Marzo 2026

Introducción

El presente trabajo final tiene como objetivo diseñar y desarrollar un conjunto de consultas SQL avanzadas aplicadas a un escenario real de comercio electrónico. Se parte de un modelo de base de datos que gestiona la información de clientes, pedidos, detalles de pedido y pagos (con metadata en formato JSONB), y se aplican los conceptos estudiados durante la asignatura de Bases de Datos Avanzado.

El trabajo se divide en dos actividades principales: la preparación del entorno (creación de tablas, inserción de datos de prueba y documentación del modelo) y el desarrollo de consultas con JOINS avanzados, incluyendo la justificación teórica del tipo de JOIN utilizado en cada caso.

Actividad 1 - Preparación del Entorno

1.1 Modelo de datos y relaciones

El modelo de datos propuesto para este sistema de comercio electrónico consta de cuatro entidades principales interrelacionadas. A continuación se describe cada tabla, sus columnas y las relaciones entre ellas:

cliente: Almacena la información básica de los clientes registrados en la plataforma. Cada cliente se identifica de forma única mediante `cliente_id` (clave primaria de tipo SERIAL). Contiene el nombre del cliente y la ciudad de origen.

pedido: Registra cada orden realizada por un cliente. Se vincula a la tabla cliente mediante la clave foránea `cliente_id`, estableciendo una relación de uno a muchos (un cliente puede tener múltiples pedidos). Incluye la fecha del pedido y su estado actual (pendiente, enviado, entregado, cancelado).

pedido_detalle: Contiene el desglose de productos dentro de cada pedido. Se relaciona con pedido mediante la clave foránea `pedido_id` en una relación de uno a muchos (un pedido puede tener múltiples líneas de detalle). Almacena el identificador del producto, la cantidad solicitada y el precio unitario.

pago: Registra los pagos asociados a cada pedido. Se vincula con pedido mediante la clave foránea `pedido_id` en una relación de uno a muchos (un pedido puede tener varios intentos de pago). Incluye la fecha del pago, el monto, un indicador booleano de confirmación y un campo metadata de tipo JSONB que almacena información flexible proveniente de pasarelas de pago externas (como el método de pago, código de transacción, pasarela utilizada, etc.).

Las relaciones del modelo se resumen de la siguiente manera:

- cliente (1) --- (N) pedido: Un cliente puede realizar muchos pedidos.
- pedido (1) --- (N) pedido_detalle: Un pedido contiene muchas líneas de detalle.
- pedido (1) --- (N) pago: Un pedido puede tener varios pagos asociados (intentos, parciales, etc.).

Campos adicionales agregados al modelo base con su justificación:

- **pedido_detalle_detalle_id (SERIAL, PK):** Se agregó una clave primaria sustituta para identificar de forma única cada línea de detalle, facilitando las operaciones de actualización y eliminación individual.

- **pago.metadata (JSONB):** Se utiliza JSONB en lugar de JSON porque permite indexación GIN, operadores de búsqueda eficientes y no almacena duplicados de claves, como se estudió en la clase sobre tipos de datos de PostgreSQL.

1.2 Script de creación de tablas

A continuación se presenta el script DDL completo para la creación de la base de datos y las tablas del modelo:

```
-- =====
-- TRABAJO FINAL - BASES DE DATOS AVANZADO
-- Estudiante: Humberto Zambrano Carreño
-- =====

-- Creación de la base de datos
CREATE DATABASE ecommerce_db;

-- Conectar a la base de datos
\c ecommerce_db;

-- =====
-- CREACIÓN DE TABLAS
-- =====

-- Tabla de clientes
CREATE TABLE cliente (
    cliente_id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    ciudad VARCHAR(80) NOT NULL
);

-- Tabla de pedidos
CREATE TABLE pedido (
    pedido_id SERIAL PRIMARY KEY,
    cliente_id INTEGER NOT NULL,
    fecha DATE NOT NULL DEFAULT CURRENT_DATE,
    estado VARCHAR(20) NOT NULL DEFAULT 'pendiente'
    CHECK (estado IN ('pendiente', 'enviado',
                     'entregado', 'cancelado')),
    CONSTRAINT fk_pedido_cliente
    FOREIGN KEY (cliente_id)
    REFERENCES cliente(cliente_id)
);

-- Tabla de detalles de pedido
CREATE TABLE pedido_detalle (
    detalle_id SERIAL PRIMARY KEY,
    pedido_id INTEGER NOT NULL,
    producto_id INTEGER NOT NULL,
    cantidad INTEGER NOT NULL CHECK (cantidad > 0),
    precio_unit NUMERIC(10,2) NOT NULL CHECK (precio_unit > 0),
    CONSTRAINT fk_detalle_pedido
    FOREIGN KEY (pedido_id)
    REFERENCES pedido(pedido_id)
);

-- Tabla de pagos con metadata JSONB
CREATE TABLE pago (
    pago_id SERIAL PRIMARY KEY,
    pedido_id INTEGER NOT NULL,
    fecha DATE NOT NULL DEFAULT CURRENT_DATE,
```

```
    monto          NUMERIC(10,2)    NOT NULL CHECK (monto > 0),
    confirmado      BOOLEAN          NOT NULL DEFAULT false,
    metadata        JSONB,
    CONSTRAINT fk_pago_pedido
        FOREIGN KEY (pedido_id)
        REFERENCES pedido(pedido_id)
);
```

Se aplicaron constraints de tipo CHECK para validar los estados permitidos del pedido, asegurar que las cantidades y precios sean positivos, y que los montos de pago sean mayores a cero. Estas restricciones garantizan la integridad de los datos, tal como se estudió en la clase sobre constraints del diseño físico.

1.3 Inserción de datos de prueba

Se insertó un conjunto de datos de prueba coherente y suficiente que cumple con los mínimos establecidos: 5 clientes, 8 pedidos, 10 detalles de pedido y 6 pagos (algunos confirmados y otros no). Los datos están diseñados para cubrir los distintos escenarios que serán consultados en la Actividad 2.

Clientes (5 registros)

```
-- =====
-- INSERCIÓN DE DATOS DE PRUEBA
-- =====

-- 5 clientes de distintas ciudades
INSERT INTO cliente (nombre, ciudad) VALUES
    ('María López', 'Guayaquil'),
    ('Carlos Mendoza', 'Quito'),
    ('Ana Torres', 'Cuenca'),
    ('Pedro Ramírez', 'Ambato'),
    ('Lucía Fernández', 'Loja');
```

Pedidos (8 registros)

```
-- 8 pedidos con distintos estados
-- Nota: Pedidos 7 y 8 no tendrán pagos asociados
INSERT INTO pedido (cliente_id, fecha, estado) VALUES
    (1, '2025-11-10', 'entregado'),
    (1, '2025-12-05', 'enviado'),
    (2, '2025-12-15', 'entregado'),
    (3, '2026-01-08', 'pendiente'),
    (4, '2026-01-20', 'enviado'),
    (5, '2026-02-01', 'cancelado'),
    (2, '2026-02-14', 'pendiente'),
    (3, '2026-03-01', 'pendiente');
```

Detalles de pedido (10 registros)

```
-- 10 detalles de pedido distribuidos entre los pedidos
INSERT INTO pedido_detalle (pedido_id, producto_id, cantidad, precio_unit) VALUES
    (1, 101, 2, 25.50),
    (1, 102, 1, 89.99),
    (2, 103, 3, 15.00),
    (3, 101, 1, 25.50),
```

```
(3, 104, 2, 42.00),
(4, 105, 1, 120.00),
(5, 102, 2, 89.99),
(5, 106, 1, 35.75),
(6, 103, 4, 15.00),
(7, 107, 1, 65.00);
```

Pagos (6 registros)

```
-- 6 pagos: algunos confirmados, otros no
-- Pedidos 7 y 8 quedarán sin pagos
-- Pedido 4 tendrá un pago NO confirmado
INSERT INTO pago (pedido_id, fecha, monto, confirmado, metadata) VALUES
(1, '2025-11-10', 140.99, true,
 '{"metodo": "tarjeta", "pasarela": "PayPal",
  "transaccion": "TXN-001", "ultimos_4": "4532"}'::jsonb),

(2, '2025-12-05', 45.00, true,
 '{"metodo": "transferencia", "pasarela": "Stripe",
  "transaccion": "TXN-002", "banco": "Banco Pichincha"}'::jsonb),

(3, '2025-12-15', 109.50, true,
 '{"metodo": "tarjeta", "pasarela": "PayPal",
  "transaccion": "TXN-003", "ultimos_4": "7891"}'::jsonb),

(4, '2026-01-10', 120.00, false,
 '{"metodo": "tarjeta", "pasarela": "MercadoPago",
  "transaccion": "TXN-004", "error": "fondos insuficientes"}'::jsonb),

(5, '2026-01-20', 215.73, true,
 '{"metodo": "tarjeta", "pasarela": "Stripe",
  "transaccion": "TXN-005", "ultimos_4": "2156"}'::jsonb),

(6, '2026-02-01', 60.00, false,
 '{"metodo": "efectivo", "pasarela": "contraentrega",
  "transaccion": "TXN-006", "nota": "cliente canceló"}'::jsonb);
```

Resumen de la distribución de datos para validar las consultas:

Pedido	Cliente	Estado	Pagos	Confirmado
1	María López	entregado	1 pago	Sí
2	María López	enviado	1 pago	Sí
3	Carlos Mendoza	entregado	1 pago	Sí
4	Ana Torres	pendiente	1 pago	No
5	Pedro Ramírez	enviado	1 pago	Sí
6	Lucía Fernández	cancelado	1 pago	No
7	Carlos Mendoza	pendiente	Sin pagos	—
8	Ana Torres	pendiente	Sin pagos	—

Actividad 2 - Consultas con JOINS Avanzados

2.1 Listar todos los pedidos, incluso aquellos sin pagos asociados

Justificación del tipo de JOIN

Para esta consulta se utiliza un LEFT JOIN (también llamado LEFT OUTER JOIN) entre las tablas pedido y pago. La razón fundamental es que se necesita listar TODOS los pedidos, independientemente de si tienen o no un pago registrado. Un INNER JOIN excluiría los pedidos sin pagos (pedidos 7 y 8 en nuestros datos), lo cual contradice el requisito explícito de incluir aquellos que no tengan pagos asociados.

El LEFT JOIN preserva todas las filas de la tabla izquierda (pedido) y las combina con las filas coincidentes de la tabla derecha (pago). Cuando no existe coincidencia, las columnas de la tabla derecha se rellenan con NULL. Adicionalmente, se incluye un LEFT JOIN con la tabla cliente para mostrar el nombre del cliente correspondiente.

Consulta SQL

```
-- =====
-- CONSULTA 2.1: Todos los pedidos (con y sin pagos)
-- JOIN utilizado: LEFT JOIN
-- Justificación: Se necesita conservar TODOS los
-- pedidos de la tabla izquierda, incluso si no
-- tienen coincidencia en la tabla pago.
-- =====

SELECT
  p.pedido_id,
  c.nombre      AS cliente,
  p.fecha       AS fecha_pedido,
  p.estado,
  pg.pago_id,
  pg.monto,
  pg.confirmado,
  CASE
    WHEN pg.pago_id IS NULL THEN 'SIN PAGO'
    ELSE 'CON PAGO'
  END
  AS estado_pago
FROM pedido p
LEFT JOIN cliente c ON p.cliente_id = c.cliente_id
LEFT JOIN pago pg ON p.pedido_id = pg.pedido_id
ORDER BY p.pedido_id;
```

Resultado esperado

La consulta devolverá 8 filas correspondientes a los 8 pedidos. Los pedidos 7 y 8 aparecerán con valores NULL en las columnas pago_id, monto y confirmado, y la columna estado_pago mostrará 'SIN PAGO'. Esto demuestra que el LEFT JOIN cumple el requisito de no excluir pedidos sin pagos.

pedido_id	cliente	estado	monto	estado_pago
1	María López	entregado	140.99	CON PAGO
2	María López	enviado	45.00	CON PAGO
3	Carlos Mendoza	entregado	109.50	CON PAGO
4	Ana Torres	pendiente	120.00	CON PAGO

5	Pedro Ramírez	enviado	215.73	CON PAGO
6	Lucía Fernández	cancelado	60.00	CON PAGO
7	Carlos Mendoza	pendiente	NULL	SIN PAGO
8	Ana Torres	pendiente	NULL	SIN PAGO

2.2 Mostrar pedidos con su cliente y el total calculado del pedido

Justificación del tipo de JOIN

Para esta consulta se utiliza un INNER JOIN entre las tablas pedido, cliente y pedido_detalle. Se elige INNER JOIN porque en este caso solo nos interesan los pedidos que efectivamente tienen líneas de detalle (y por tanto un total calculable) y que pertenecen a un cliente registrado. Un pedido sin detalles no tendría un total significativo que mostrar, por lo que es correcto excluirllos.

El total del pedido se calcula multiplicando la cantidad por el precio unitario de cada línea de detalle y sumando los resultados mediante la función de agregación SUM(), agrupando con GROUP BY por los campos del pedido y del cliente.

Consulta SQL

```
-- =====
-- CONSULTA 2.2: Pedidos con cliente y total calculado
-- JOIN utilizado: INNER JOIN
-- Justificación: Solo interesan pedidos que tengan
-- detalles (para calcular el total) y que estén
-- vinculados a un cliente existente. No se necesitan
-- pedidos sin detalles ni clientes sin pedidos.
-- =====

SELECT
    p.pedido_id,
    c.nombre           AS cliente,
    c.ciudad,
    p.fecha,
    p.estado,
    COUNT(d.detalle_id) AS lineas_detalle,
    SUM(d.cantidad * d.precio_unit) AS total_pedido
FROM pedido p
INNER JOIN cliente c      ON p.cliente_id = c.cliente_id
INNER JOIN pedido_detalle d ON p.pedido_id = d.pedido_id
GROUP BY p.pedido_id, c.nombre, c.ciudad, p.fecha, p.estado
ORDER BY total_pedido DESC;
```

Resultado esperado

La consulta devolverá solo los pedidos que tienen al menos una línea de detalle. El pedido 8 no aparecerá porque no tiene registros en pedido_detalle. Los totales se calculan dinámicamente a partir de cantidad × precio_unit.

pedido_id	cliente	ciudad	líneas	total_pedido
5	Pedro Ramírez	Ambato	2	215.73
1	María López	Guayaquil	2	140.99

4	Ana Torres	Cuenca	1	120.00
3	Carlos Mendoza	Quito	2	109.50
7	Carlos Mendoza	Quito	1	65.00
6	Lucía Fernández	Loja	1	60.00
2	María López	Guayaquil	1	45.00

2.3 Identificar pedidos que no tienen pagos confirmados

Justificación del tipo de JOIN

Para esta consulta se utiliza un LEFT JOIN entre pedido y pago, combinado con una condición de filtro que identifica dos situaciones: pedidos que no tienen ningún pago asociado, y pedidos cuyos pagos existen pero no están confirmados.

Se elige LEFT JOIN porque es necesario conservar todos los pedidos en el resultado, incluyendo aquellos que no tienen ningún registro en la tabla pago. Si usáramos un INNER JOIN, los pedidos sin pagos quedarían excluidos del resultado, y precisamente esos son parte del conjunto que buscamos identificar. La condición de filtro en el WHERE (pg.pago_id IS NULL OR pg.confirmando = false) captura ambos escenarios: la ausencia total de pago y la existencia de pagos no confirmados.

Consulta SQL

```
-- =====
-- CONSULTA 2.3: Pedidos sin pagos confirmados
-- JOIN utilizado: LEFT JOIN
-- Justificación: Se necesita incluir pedidos que NO
--   tienen ningún registro en la tabla pago (NULL),
--   además de aquellos con pagos no confirmados.
--   Un INNER JOIN excluiría los pedidos sin pagos.
-- =====

SELECT
  p.pedido_id,
  c.nombre      AS cliente,
  p.fecha       AS fecha_pedido,
  p.estado,
  pg.pago_id,
  pg.monto,
  pg.confirmando,
  CASE
    WHEN pg.pago_id IS NULL THEN 'Sin pago registrado'
    WHEN pg.confirmando = false THEN 'Pago no confirmado'
  END          AS motivo
FROM pedido p
LEFT JOIN cliente c ON p.cliente_id = c.cliente_id
LEFT JOIN pago pg ON p.pedido_id = pg.pedido_id
WHERE pg.pago_id IS NULL
      OR pg.confirmando = false
ORDER BY p.pedido_id;
```

Resultado esperado

La consulta identificará los pedidos que requieren atención: aquellos con pagos rechazados o no confirmados y aquellos que aún no tienen ningún intento de pago.

pedido_id	cliente	estado	pago_id	motivo
4	Ana Torres	pendiente	4	Pago no confirmado
6	Lucía Fernández	cancelado	6	Pago no confirmado
7	Carlos Mendoza	pendiente	NULL	Sin pago registrado
8	Ana Torres	pendiente	NULL	Sin pago registrado

Consulta alternativa con NOT EXISTS

Una forma alternativa y equivalente de resolver este problema es usar NOT EXISTS para encontrar pedidos que no tienen ningún pago confirmado. Esta técnica es útil cuando solo interesa saber si existe o no un pago confirmado, sin necesidad de mostrar los detalles del pago:

```
-- Alternativa con NOT EXISTS
-- Encuentra pedidos donde no existe ningún pago confirmado
SELECT
    p.pedido_id,
    c.nombre      AS cliente,
    p.estado
FROM pedido p
INNER JOIN cliente c ON p.cliente_id = c.cliente_id
WHERE NOT EXISTS (
    SELECT 1 FROM pago pg
    WHERE pg.pedido_id = p.pedido_id
    AND pg.confirmado = true
)
ORDER BY p.pedido_id;
```

Conclusiones

A lo largo de este trabajo final se aplicaron de forma práctica los conceptos avanzados de SQL estudiados durante el curso de Bases de Datos Avanzado. La resolución del caso de comercio electrónico permitió demostrar la importancia de elegir correctamente el tipo de JOIN según el requisito funcional de cada consulta.

Se evidenció que el LEFT JOIN es esencial cuando se necesita preservar todos los registros de la tabla principal, como en la consulta de pedidos con y sin pagos. Por otra parte, el INNER JOIN es la opción adecuada cuando solo se requieren registros que tengan correspondencia en ambas tablas, como al calcular el total de un pedido a partir de sus detalles.

El uso del tipo de dato JSONB para la metadata de pagos demostró la flexibilidad de PostgreSQL para manejar datos semi-estructurados provenientes de fuentes externas como pasarelas de pago, sin sacrificar la capacidad de consulta ni la eficiencia, tal como se revisó en las clases sobre tipos de datos del diseño físico.

Además, la aplicación de constraints (CHECK, NOT NULL, FOREIGN KEY) en la creación de las tablas garantiza la integridad de los datos desde el nivel de diseño físico, previniendo los errores típicos de implementación estudiados en clase, como la existencia de registros huérfanos, datos nulos no deseados o estados inválidos.

Finalmente, la correcta comprensión de los tipos de JOIN no solo permite escribir consultas que devuelvan resultados correctos, sino que también tiene implicaciones directas en el rendimiento, ya que el motor de base de datos elige diferentes algoritmos de ejecución (Nested Loop, Hash Join, Merge Join) según el tipo de unión y la presencia de índices.

Anexo - Script SQL Completo

A continuación se presenta el script completo unificado, listo para ejecutar en PostgreSQL:

```
-- =====
-- TRABAJO FINAL - BASES DE DATOS AVANZADO
-- Estudiante: Humberto Zambrano Carreño
-- Materia: Base de Datos Avanzados B
-- CEU Felipe Villanueva
-- =====

-- =====
-- PARTE 1: CREACIÓN DE TABLAS
-- =====

CREATE DATABASE ecommerce_db;
\c ecommerce_db;

CREATE TABLE cliente (
    cliente_id SERIAL PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    ciudad VARCHAR(80) NOT NULL
);

CREATE TABLE pedido (
    pedido_id SERIAL PRIMARY KEY,
    cliente_id INTEGER NOT NULL,
    fecha DATE NOT NULL DEFAULT CURRENT_DATE,
    estado VARCHAR(20) NOT NULL DEFAULT 'pendiente'
    CHECK (estado IN ('pendiente', 'enviado',
                      'entregado', 'cancelado')),
    CONSTRAINT fk_pedido_cliente
    FOREIGN KEY (cliente_id)
    REFERENCES cliente(cliente_id)
);

CREATE TABLE pedido_detalle (
    detalle_id SERIAL PRIMARY KEY,
    pedido_id INTEGER NOT NULL,
    producto_id INTEGER NOT NULL,
    cantidad INTEGER NOT NULL CHECK (cantidad > 0),
    precio_unit NUMERIC(10,2) NOT NULL CHECK (precio_unit > 0),
    CONSTRAINT fk_detalle_pedido
    FOREIGN KEY (pedido_id)
    REFERENCES pedido(pedido_id)
);

CREATE TABLE pago (
    pago_id SERIAL PRIMARY KEY,
    pedido_id INTEGER NOT NULL,
    fecha DATE NOT NULL DEFAULT CURRENT_DATE,
    monto NUMERIC(10,2) NOT NULL CHECK (monto > 0),
    confirmado BOOLEAN NOT NULL DEFAULT false,
    metadata JSONB,
    CONSTRAINT fk_pago_pedido
    FOREIGN KEY (pedido_id)
    REFERENCES pedido(pedido_id)
);

-- =====
-- PARTE 2: INSERCIÓN DE DATOS
```

```
-- =====

INSERT INTO cliente (nombre, ciudad) VALUES
  ('María López', 'Guayaquil'),
  ('Carlos Mendoza', 'Quito'),
  ('Ana Torres', 'Cuenca'),
  ('Pedro Ramírez', 'Ambato'),
  ('Lucía Fernández', 'Loja');

INSERT INTO pedido (cliente_id, fecha, estado) VALUES
  (1, '2025-11-10', 'entregado'),
  (1, '2025-12-05', 'enviado'),
  (2, '2025-12-15', 'entregado'),
  (3, '2026-01-08', 'pendiente'),
  (4, '2026-01-20', 'enviado'),
  (5, '2026-02-01', 'cancelado'),
  (2, '2026-02-14', 'pendiente'),
  (3, '2026-03-01', 'pendiente');

INSERT INTO pedido_detalle (pedido_id, producto_id, cantidad, precio_unit) VALUES
  (1, 101, 2, 25.50),
  (1, 102, 1, 89.99),
  (2, 103, 3, 15.00),
  (3, 101, 1, 25.50),
  (3, 104, 2, 42.00),
  (4, 105, 1, 120.00),
  (5, 102, 2, 89.99),
  (5, 106, 1, 35.75),
  (6, 103, 4, 15.00),
  (7, 107, 1, 65.00);

INSERT INTO pago (pedido_id, fecha, monto, confirmado, metadata) VALUES
  (1, '2025-11-10', 140.99, true,
    '{"metodo": "tarjeta", "pasarela": "PayPal",
      "transaccion": "TXN-001", "ultimos_4": "4532"}'::jsonb),
  (2, '2025-12-05', 45.00, true,
    '{"metodo": "transferencia", "pasarela": "Stripe",
      "transaccion": "TXN-002", "banco": "Banco Pichincha"}'::jsonb),
  (3, '2025-12-15', 109.50, true,
    '{"metodo": "tarjeta", "pasarela": "PayPal",
      "transaccion": "TXN-003", "ultimos_4": "7891"}'::jsonb),
  (4, '2026-01-10', 120.00, false,
    '{"metodo": "tarjeta", "pasarela": "MercadoPago",
      "transaccion": "TXN-004", "error": "fondos insuficientes"}'::jsonb),
  (5, '2026-01-20', 215.73, true,
    '{"metodo": "tarjeta", "pasarela": "Stripe",
      "transaccion": "TXN-005", "ultimos_4": "2156"}'::jsonb),
  (6, '2026-02-01', 60.00, false,
    '{"metodo": "efectivo", "pasarela": "contraentrega",
      "transaccion": "TXN-006", "nota": "cliente canceló"}'::jsonb);

-- =====
-- PARTE 3: CONSULTAS AVANZADAS
-- =====

-- CONSULTA 2.1: Todos los pedidos (con y sin pagos)
-- JOIN: LEFT JOIN (preserva todos los pedidos)
SELECT
  p.pedido_id,
  c.nombre      AS cliente,
  p.fecha       AS fecha_pedido,
  p.estado,
```

```
pg.pago_id,
pg.monto,
pg.confirmado,
CASE
    WHEN pg.pago_id IS NULL THEN 'SIN PAGO'
    ELSE 'CON PAGO'
END
AS estado_pago
FROM pedido p
LEFT JOIN cliente c ON p.cliente_id = c.cliente_id
LEFT JOIN pago pg ON p.pedido_id = pg.pedido_id
ORDER BY p.pedido_id;

-- CONSULTA 2.2: Pedidos con cliente y total calculado
-- JOIN: INNER JOIN (solo pedidos con detalles)
SELECT
    p.pedido_id,
    c.nombre AS cliente,
    c.ciudad,
    p.fecha,
    p.estado,
    COUNT(d.detalle_id) AS lineas_detalle,
    SUM(d.cantidad * d.precio_unit) AS total_pedido
FROM pedido p
INNER JOIN cliente c ON p.cliente_id = c.cliente_id
INNER JOIN pedido_detalle d ON p.pedido_id = d.pedido_id
GROUP BY p.pedido_id, c.nombre, c.ciudad, p.fecha, p.estado
ORDER BY total_pedido DESC;

-- CONSULTA 2.3: Pedidos sin pagos confirmados
-- JOIN: LEFT JOIN (incluye pedidos sin ningún pago)
SELECT
    p.pedido_id,
    c.nombre AS cliente,
    p.fecha AS fecha_pedido,
    p.estado,
    pg.pago_id,
    pg.monto,
    pg.confirmado,
    CASE
        WHEN pg.pago_id IS NULL THEN 'Sin pago registrado'
        WHEN pg.confirmado = false THEN 'Pago no confirmado'
    END
    AS motivo
FROM pedido p
LEFT JOIN cliente c ON p.cliente_id = c.cliente_id
LEFT JOIN pago pg ON p.pedido_id = pg.pedido_id
WHERE pg.pago_id IS NULL
    OR pg.confirmado = false
ORDER BY p.pedido_id;
```